As you've seen in earlier chapters, middleware often handles cross-cutting concerns or narrowly defined requests, such as requests for files. For requirements that fall outside these functions or that have many external dependencies, a more robust framework is required. Razor Pages (and/or ASP.NET Core MVC) can provide this framework, allowing interaction with your application's core business logic and the generation of a UI. It handles everything from mapping the request to an appropriate page handler (or controller action method) to generating the HTML response.

In the traditional description of the MVC design pattern, there's only a single type of model, which holds all the non-UI data and behavior. The controller updates this model as appropriate and then passes it to the view, which uses it to generate a UI.

One of the problems when discussing MVC is the vague and ambiguous terms that it uses, such as *controller* and *model. Model* in particular is such an overloaded term that it's often difficult to be sure exactly what it refers to; is it an object, a collection of objects, or an abstract concept? Even ASP.NET Core uses the word *model* to describe several related but different components, as you'll see later in this chapter.

13.4.1 Directing a request to a Razor Page and building a binding model

The first step when your app receives a request is routing the request to an appropriate Razor Page handler in the routing middleware. Let's think again about the category to-do list page in listing 13.5. On that page, you're displaying a list of items that have a given category label. If you're looking at the list of items with a category of Simple, you'd make a request to the `/category/Simple` path.